HOW THE PROJECT WORKS

# DDROID livestock water point

*A stationary controller fills a drinking trough, latches faults, and records how much water each collared cow drank. Gazebo is the yard. ROS 2 carries status and operator commands.*

*22 October 2026 n02528843x@students.nust.ac.zw*

## 1. What the system is

DDROID is a prototype of a smart livestock water point. One machine sits beside a storage tank and a concrete trough. Float switches tell it whether the trough is low, full, or over the safety line, and whether the storage tank still has water. A low-voltage pump moves water from the tank into the trough. Cattle drink from the trough when the pump is off.

The controller is an ESP32-style state machine: STOPPED, IDLE, REFILLING, MANUAL, and FAULT. It does not depend on a dashboard. If the link to an operator is lost, the machine on the bench still owns the pump. ROS 2 only publishes that state and accepts commands. Gazebo draws the same numbers as moving water, lamps, and cattle.

A second function sits on top of the watering cycle. Each animal wears a collar. A camera at the trough reads the collar colour of the cows that are drinking, and the litres that left the trough are shared across those collars. The same collar position is the GPS fix.

## 2. How a run is organised

Three programs cooperate. They share one clock of 0.1 second steps.

- **Plant and controller** (`simulation/ddroid_control.py`). The plant holds the two water volumes and moves them. The controller reads the switches and decides whether the pump may run.
- **Herd ledger** (`simulation/ddroid_track.py`). After each step it takes the litres just drunk and credits them to the collars the camera picked. It also stores each collar’s x, y.
- **View** (`simulation/run_sim.py`). If Gazebo is running, this moves the water slices, lamps, markers, and cattle, and reads the trough-camera image.

The supervised story in `simulation/ddroid_demo.py` calls the controller, then the plant, then the herd ledger, then the view. The ROS 2 node does the same work and publishes the result. Headless runs skip the view. The litre totals are still recorded.

## 3. The water plant

There are two vessels. The storage tank holds 18 litres and starts at 12.3 litres. The trough holds 10 litres and starts at 1.2 litres, which is below the low switch, so the first arm starts a refill. A top-up command sets the tank to 14.0 litres.

When the controller asks for the pump, and the high-cutoff switch is dry, water moves from the tank to the trough at 0.40 litres per second. The move stops early if the tank is empty, the trough is full to its capacity, or the level reaches the cutoff. Reaching the cutoff also drops pump drive on that same step, even if software still asks for the pump.

When the pump is off and the trough is not empty, the herd drinks at 0.32 litres per second. That single rate is the whole herd, not one cow. The amount removed on a step is saved as `last_drunk_l`. While the pump is on, that value is zero, so no cow is credited.

## 4. Float switches

Switch levels are fractions of each vessel. The names match the states the controller reports. The high cutoff is separate from the high switch: it is a hardware interlock, and it is not debounced.

| Switch | Vessel | Threshold | Litres | Meaning when wet |
|---|---|---|---|---|
| Trough low | Trough | 30% | 3.0 L | Above LOW |
| Trough high | Trough | 75% | 7.5 L | FULL, if low is also wet |
| Trough cutoff | Trough | 92% | 9.2 L | HIGH CUTOFF, immediate |
| Supply low | Tank | 20% | 3.6 L | Supply AVAILABLE |

The debounced trough state is one of four words:

- **LOW** when the low switch is dry.
- **ADEQUATE** when the low switch is wet and the high switch is dry. Drinking through this band does not start the pump.
- **FULL** when both trough switches are wet. A refill stops and the mode becomes IDLE.
- **SENSOR FAULT** when the high switch is wet and the low switch is dry. That combination is impossible for a real water surface, so it latches a fault.

Low, high, and supply changes are accepted only after two samples taken one second apart. A single splash does not change the state. The cutoff switch is read every step and latches HIGH CUTOFF immediately. In Gazebo each vessel is drawn as 16 horizontal slices, so the visible surface steps with the litres.

## 5. The controller

The machine starts in STOPPED with the pump off. `arm` is accepted only from STOPPED, and only when the inputs are safe: cutoff dry, supply available, no sensor fault, and no latched fault. If the trough is LOW, arm enters REFILLING and the pump runs. Otherwise arm enters IDLE and the pump stays off.

REFILLING runs until the trough state is FULL, then returns to IDLE and stops the pump. If a refill lasts 60 seconds it latches PUMP TIMEOUT. From IDLE, the next time the debounced state is LOW, and the inputs are still safe, the pump starts again by itself. ADEQUATE never starts a refill.

`manual` is also only from STOPPED, with the same safety checks, and it is refused if the trough is already FULL. MANUAL runs the pump for at most 15 seconds, or until FULL, and then returns to STOPPED rather than IDLE. `stop` from a normal mode returns to STOPPED and clears the run timers. From FAULT it only forces the pump off; the fault stays latched.

`acknowledge` marks a fault as seen. It does not clear it. `reset` is accepted only in FAULT, and only after the cause has gone:

- LOW SUPPLY cannot be reset while the supply switch is still dry.
- SENSOR FAULT cannot be reset while the high switch is wet and the low switch is dry.
- HIGH CUTOFF cannot be reset while the cutoff switch is still wet.

A successful reset returns the machine to STOPPED with the pump off. Another arm is required before it will idle or refill.

## 6. Faults

| Fault | When it latches | Pump |
|---|---|---|
| HIGH CUTOFF | Cutoff switch is wet. Same step, no debounce. | Drive removed immediately |
| LOW SUPPLY | Supply switch is dry, after debounce. | Off |
| SENSOR FAULT | High wet and low dry, after debounce. | Off |
| PUMP TIMEOUT | REFILLING has run for 60 seconds. | Off |

Any fault forces the software pump request off. The plant applies a second check: even if something outside the state machine forces the pump, a wet cutoff removes hardware drive and clamps the trough at the cutoff level.

## 7. The yard

The Gazebo world is `simulation/worlds/ddroid_water_point.sdf`. Props are not written out in full inside that file. Each prop is a model under `simulation/models`, and the world places it with an include of the form `model://name`. `scripts/launch_gui.sh` adds each group folder to `GZ_SIM_RESOURCE_PATH`, which is how Gazebo finds them; Gazebo 10 resolves a plain relative path from the directory it was started in, not from the world file.

| Folder | What it holds |
|---|---|
| Trees | A flat-topped acacia, used for the shade tree in the kraal and four outside it. |
| Cattle | One rounded cow model per coat. Each includes that animal’s collar. |
| Kraal | A gum-pole post, three rail lengths for the three-rail fence, and a gate leaf. |
| Yard | Grass, the dirt pad, trampled mud at the trough, the gate track, rocks, and grass tufts. |
| Water | Storage tank, trough, plinth, pipe, and one water-slice model per vessel. |
| Controls | Pump, controller cabinet, solar panel, pump and mode lamps, and float-switch markers. |
| Sign | The status board under a small roof, on two posts. |
| Cameras | The drink camera on a post at the east end of the trough. |

`python3 simulation/build_world.py` rewrites both the model folders and the world. Geometry numbers live in `ddroid_scene.py`, so the drawing and the controller stay on the same trough. The builder refuses to write a world in which any prop other than a collar has a colour the drink camera would take for a collar tag.

An overview camera looks down on the kraal from the south-east. A sign at the south gate shows mode, pump, trough state and litres, supply state and litres, whether the herd is drinking, and the fault. A second text board beside the drink camera lists litres and the collar position for C1 to C5. The mode lamp sits on top of the controller cabinet beside the pump, and the pump lamp sits above the pump. Both are glowing spheres that are parked underground when they are off, and moved into place when they are on.

## 8. Collars, the camera, and GPS

Nine animals wear collars. Five drink (C1 to C5). Four walk closed paths inside the kraal (W1 to W4) and stay clear of the trough. The collar is a band and a coloured plate on the right side of the neck, plus a small puck on top. Nothing is pierced or implanted. Each plate has its own saturated colour so it can be told apart from hides, dirt, concrete, and water.

The drink camera stands at the east end of the trough and looks west along the south lip. Its view is a box around collar height at that lip. While the herd is drinking, two of the five cows stand in that box. The pair changes every 6 seconds, rotating through the herd, so the totals come out different. Cows that are waiting stand back at their rest poses. The walkers keep moving the whole time.

GPS for every animal is the world position of that collar plate, in metres. It is published even when the animal is not drinking. On the board and on `/ddroid/gps` the form is `cow_1:0.27,-0.23`.

## 9. How intake is measured

There is no flow meter on each mouth. Intake is the drop in the trough, shared across the collars that are actually at the lip.

Each 0.1 second step, if the pump is off and the trough still has water, the plant removes up to 0.32 × 0.1 litres and stores that exact drop. The herd ledger then chooses who was drinking:

- The latest drink-camera frame is scanned for collar colours. A colour counts when enough pixels are close to that tag and clearly saturated, so grey concrete and brown hides are ignored.

- A cow is credited only if that colour was seen **and** the collar is inside the camera's view box. A colour reported for an animal elsewhere in the kraal is ignored.
- The step's litres are divided equally by the number of collars that pass both checks.
- If the camera frame is missing, or none of its colours match a collar at the lip, the litres are shared by every collar that is physically inside the view. The log then says `tag` instead of `camera`.

Two cows at the lip for one second therefore each receive 0.16 litres. The running total is `/ddroid/intake`, for example `cow_1:1.500,cow_2:0.540,...`, including the walkers at 0. A top-down clip of this behaviour is `media/intake_demo.mp4`. Regenerate it with `python tools/render_intake_clip.py`.

## 10. The supervised demonstration

`ddroid_demo.py` drives one full story from controller events, not from a fixed script of seconds. Debounce and pump timing can change without breaking the sequence. The steps are:

- Startup is STOPPED and the pump is off. The trough is already LOW.
- Arm starts REFILLING. The refill runs until FULL, then the mode is IDLE.
- The herd drinks through ADEQUATE. The pump stays off. The next LOW starts another refill.
- That second refill crosses the supply-low switch and latches LOW SUPPLY. Reset is rejected while the supply is still low.
- The operator tops the tank up to 14 litres. After the supply switch is wet, reset returns STOPPED and arm enters IDLE.
- The trough is refilled again.
- A safety drill forces the pump on past the high switch. The independent cutoff latches HIGH CUTOFF and removes pump drive.
- After the cutoff dries, reset returns STOPPED and arm leaves the machine IDLE. The pump stays off until the trough is LOW again.

`python3 test_control.py` runs that story without Gazebo and checks the event order, the rejected reset, both faults, and the per-cow split of the litres.

## 11. ROS 2

The package `ddroid_water_point` does not reimplement the state machine. It imports the simulation modules and publishes their state. Two executables are installed: `ddroid_demo` runs the story, and `ddroid_controller` waits for operator commands. Pass `--gazebo` to either one to drive a Gazebo session that is already running on the `ddroid` partition.

| Topic | Type | Contents |
|---|---|---|
| /ddroid/status | String | One line: mode, pump, both vessels, fault |
| /ddroid/mode | String | STOPPED, IDLE, REFILLING, MANUAL, or FAULT |
| /ddroid/pump | Bool | True while hardware pump drive is on |
| /ddroid/fault | String | Latched fault, or empty |
| /ddroid/trough_state | String | LOW, ADEQUATE, FULL, or SENSOR FAULT |
| /ddroid/supply_state | String | AVAILABLE or LOW |
| /ddroid/trough_litres | Float32 | Trough volume |
| /ddroid/supply_litres | Float32 | Tank volume |
| /ddroid/intake | String | Litres credited to each collar |
| /ddroid/gps | String | Collar x, y in metres for each animal |
| /ddroid/command | String | arm, stop, reset, acknowledge, manual, topup |

Commands are accepted only by the controller node, not during the scripted demonstration. `topup` sets the tank to 14 litres. Unknown text is rejected with a short reason.

## 12. How to run it

This copy runs on Windows with WSL2. The guest is Ubuntu 26.04, Gazebo is Jetty (`gz sim` 10), and ROS 2 is Lyrical. Gazebo uses the NVIDIA GeForce GTX 1650 through WSLg. The launch script selects that adapter so the Intel chip is left unused. Install steps for a new machine are in the repository README.

Controller check, from simulation/ inside Ubuntu:

```
cd simulation
python3 test_control.py
```

Open the yard and leave that terminal running:

```
bash scripts/launch_gui.sh
```

In a second terminal, from the repository root, run the story into that window:

```
bash scripts/run_ros_demo.sh --gazebo
```

Omit --gazebo for the same story with no window. Watch the middleware from a third terminal:

```
source /opt/ros/lyrical/setup.bash
source ros2_ws/install/setup.bash
ros2 topic echo /ddroid/status
```

Operator commands, with Gazebo already running:

```
source /opt/ros/lyrical/setup.bash
source ros2_ws/install/setup.bash
export GZ_PARTITION=ddroid
export DDROID_SIM="$PWD/simulation"
ros2 run ddroid_water_point ddroid_controller --gazebo
```

```
ros2 topic pub --once /ddroid/command std_msgs/msg/String "{data: arm}"
```

`bash scripts/verify_ros.sh` rebuilds the package, reruns the controller checks, and prints one status message.

## 13. Where the code lives

| File | Role |
|---|---|
| simulation/ddroid_scene.py | Litres, switch levels, poses, collar colours, camera view. |
| simulation/ddroid_control.py | Plant and the five-mode state machine. |
| simulation/ddroid_track.py | Camera colour match, intake ledger, GPS. |
| simulation/ddroid_demo.py | The supervised watering story. |
| simulation/run_sim.py | Moves Gazebo and reads the drink camera. |
| simulation/build_world.py | Writes the model folders and the world. |
| simulation/test_control.py | Headless checks of the story and the intake split. |
| ros2_ws/.../demo_node.py | Publishes state and accepts commands. |
| scripts/ | Install, launch, demo, and verify shell scripts. |
| tools/render_intake_clip.py | Writes media/intake_demo.mp4. |
| tools/build_guide.py | Writes this guide. |
| tools/snapshot_world.py | Saves one headless frame from each camera. |
| docs/DDROID_SRS.pdf | The group requirements specification. |

---

*The watering rules are the device. The camera and the GPS collar explain which animal drank. Both use the same litres, so the trough level and the sum of the credited intakes stay in step.*